

Analysing tradeoffs of CNN on MNIST

Carl Petter Arthur, 25142201x

1. Motivation

Real world Artificial Intelligence of Things (AIoT) systems generate tons of sensor data that must be processed in real-time in order to make good decisions. A big challenge in these systems is that information is highly time sensitive, meaning that if the data is too old, it might simply not be relevant anymore. Making decisions based on old data could hurt the system, while on the other hand making decisions based on inaccurate but fresh data can also hurt the system. This creates a trade-off, one that will be analysed in this project.

To understand the problem deeper, the different factors should be described. First of all, the system suffers from **limited resources**. The edge device typically holds less computational power and memory compared to cloud servers. This means that calculating on the cloud server is faster, however it introduces a **communication latency**. Communication latency means that when the system offloads tasks to the cloud server, it introduces network delays that can negatively impact the freshness of the data. Also, there's the factor of **capacity**, when the data arrival rate exceeds the system's processing rate, the system must somehow adapt to prevent queue overflows and an increased latency.

The motivation of this project is therefore to explore and evaluate the trade-offs between accuracy, computation cost and latency. Instead of focusing entirely on high precision, this work will create a prototype that uses model compression (pruning and quantization) and distributed deployment (i.e. cloud offloading) to maintain system quality. By evaluating these strategies in different scenarios, such as overloaded and offloaded, I will demonstrate how AIoT trades accuracy with latency, and which factors affect it. The aim is therefore not to build a perfect AIoT, but instead one that can be analysed under different scenarios to build an understanding of the subject in question.

2. System Design

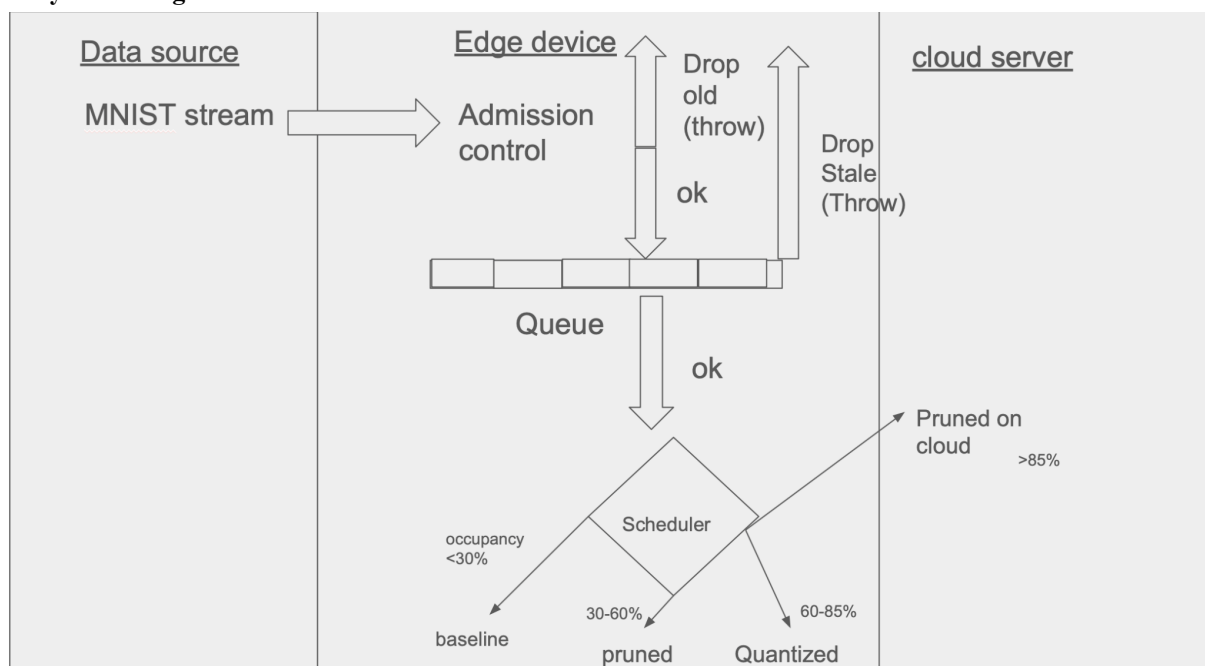


Figure 1, The diagram illustrates the data flow from MNIST sensor stream through the Edge devices admission control and queue. The scheduler then selects the execution path (baseline, pruned, quantized or cloud offloading) based on the occupancy of the queue.

This system is designed to balance streaming data by balancing edge device execution in different ways with cloud computing. The main idea is to run on edge device as long as possible, and using the cloud server only when it's needed since this introduces the communication latency.

Architectural overview

As illustrated in Figure 1, the system lets data flow from source through an admission controller, after which a scheduler determines the execution path based on the queue.

Admission control

To handle overload (i.e when arrival rate > processing rate), the system uses an admission control strategy. Since the system maintains a big data queue, when a sample arrives while the queue is full, a decision has to be made. The system contains two ways to handle this which both will be studied:

- Drop-New: If the queue is full, the incoming sample is discarded. This could lead to stale data occupying the queue while new, relevant data is dropped.
- Drop-Old: If the queue is full, the oldest data is thrown, making room for new more relevant data. This ensures the system prioritises newer data, minimizing the Age of Information (AoI).

Lastly, a Drop-Stale mechanism is used during the processing phase. This means that before the model is run, the sample is checked. If its latency exceeds the deadline, it's discarded to avoid wasting computation on irrelevant data.

AI Scheduler

The AI scheduler then uses a threshold based policy. By calculating the Occupancy Ratio of the queue: $\text{Current Queue Length} / \text{Max Queue Size}$. The policy is divided into 4 operational modes:

Mode	Occupancy	Strategy	Goal
High precision	<30%	Edge, Baseline model	High accuracy
Efficient	30-60%	Edge, pruned model	Reduce compute latency
Aggressive	60-80%	Edge, quantized model	Minimize execution time
Distributed	>85%	Cloud offloading, pruned model	Handle overload

Communication Delay

When the model enters the distributed mode (cloud offloading), the communication latency is important to take into account. This represents the time taken to transmit the MNIST image and receive the classification result back to the edge device.

3. Implementation

NOTE: if you're running the code on a device other than MAC, please change the engine in codeblock 2 to "fbgemm" as stated in the code.

All code was written in Python. The system was implemented using PyTorch framework, using its built-in libraries for model compression and data handling.

Baseline Model and training

The baseline CNN model used in this project is directly taken from the official PyTorch MNIST example repository [1]. The network consists of two convolutional layers, followed by two fully connected layers. The model was trained on the MNIST training dataset for one epoch using Adadelta optimizer [2].

Model Compression

Two optimized versions of the baseline model were generated:

Pruned model: Structured L2-norm pruning was applied to the convolutional layers to accelerate computation, and unstructured pruning on linear layers to reduce memory footprint, using torch.nn.utils.prune [3]. By removing 30% of the filters and reducing the total MAC operations, the model's inference time and size should be reduced with a slight trade-off in accuracy.

Quantized Model: Post-training quantization was implemented via torch.ao.quantization. This converts weights of the linear layers from float32 to int8, reducing memory usage which should lead to a small loss in accuracy.

Simulation of environment

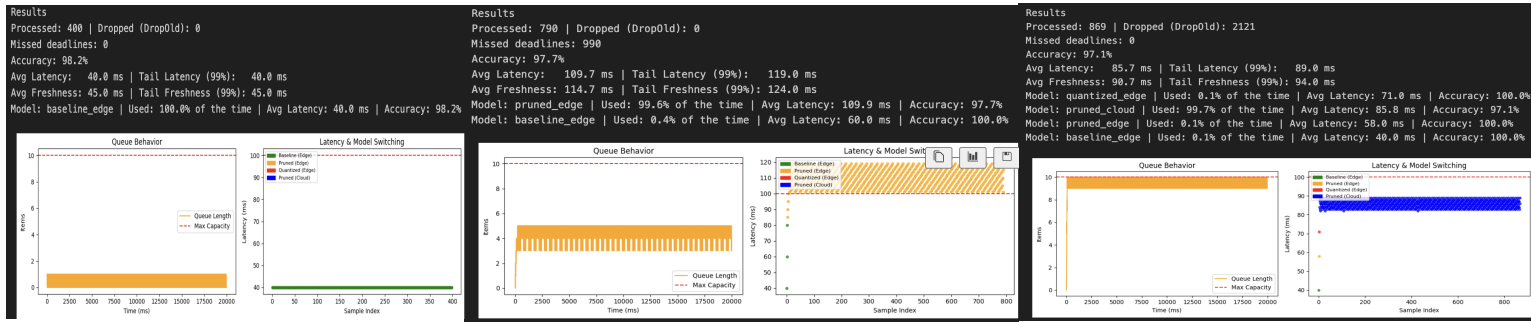
To evaluate the system, a time simulation was implemented. It runs a 1 ms resolution over a total duration of 20000 ms, tracking the AoI for every data sample. By feeding MNIST test images into the system, a datastream generator is created.

The system settings are: Maximum Queue Size: $N=10$, Decision Deadline: 100 ms, Network Latency: 20 ms. Further, the computation times were simulated as follows:
Edge Device: Baseline (40 ms), Pruned (25 ms), Quantized (20 ms).
Cloud Server: Baseline (5 ms), Pruned (3 ms), Quantized (2 ms).
There is also a sensor data arrival rate, which will be adjusted in the results section to deepen the analysis.

The evaluation metrics are latency, tail latency, average freshness, tail freshness, total dropped samples and overall system accuracy as well as MAC's of each model variant.

4. Experiments & Results

Three cases are selected for this results section, which demonstrate how the system acts during different loadings.



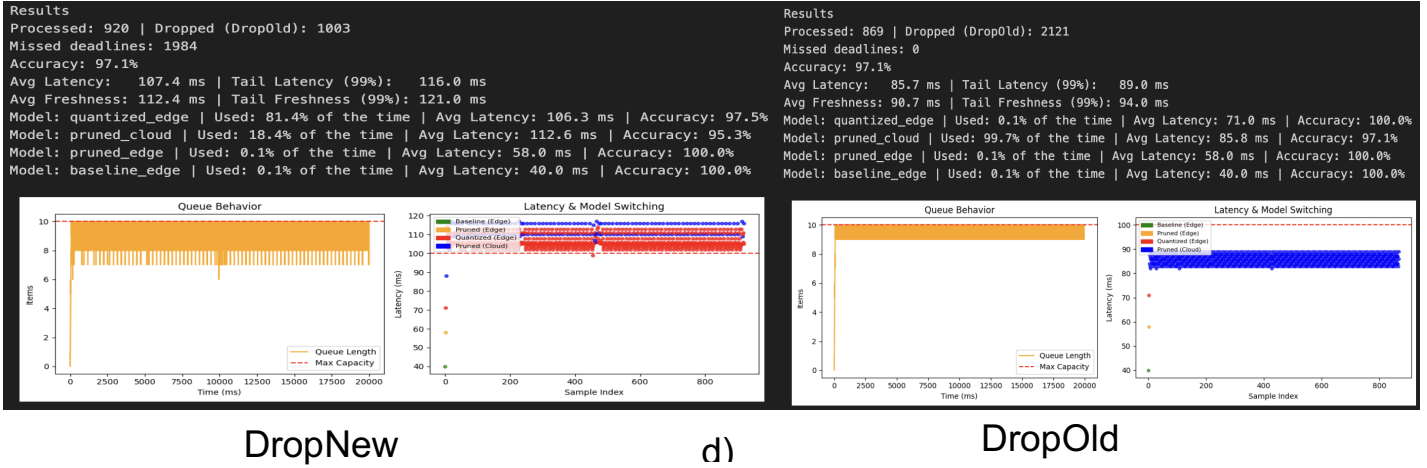
b

Low data arrival rate (fps=20), as demonstrated in figure a) the baseline model is always running on edge device, and the queue never gets loaded. Accuracy is 98.2%, average latency is 40.0 ms, and tail latency is 40.0 ms, with 0 missed deadlines and 0 dropped samples. Average data freshness is 45.0 ms and tail freshness is 45.0 ms.

Overload (fps=50), as shown in figure b), the system runs the pruned model on the edge device, misses the deadline while the queue remains stable between 4-6 items. Accuracy is 97.7% and no items are dropped. Average latency is 109.7 ms and tail latency is 119.0 ms, with 990 missed deadlines and 790 processed samples. Average data freshness is 114.7 ms and tail freshness is 124.0 ms.

Highly overload (fps=150), as shown in figure c) the system runs the pruned model on cloud, latency is under deadline (85.7 ms) while queue is stable between 9-10. Accuracy dropped to 97.1%, processing 869 samples and dropping 2121 (70.9% drop rate), with 0 missed deadlines. Average data freshness is 90.7 ms and tail freshness is 94.0 ms.

As previously mentioned, the queue was implemented with two different overflow policies, DropOld and DropNew, to compare their performance in this project. Specifically the highly overloaded scenario (150 FPS) will be studied, which is demonstrated in figure D).



The results show that DropOld performs better, staying below the latency deadline while dropping 2121 samples compared to the 1003 samples dropped by DropNew. While dropping more items, DropOld holds a lower latency. The DropNew implementation fluctuates between a quantized model on edge and a pruned model on Cloud, while the DropOld implementation always uses the pruned model on cloud. Something worth mentioning is that the accuracy is the same(97.1%), DropNew while also processes more samples (920) compared to when DropOld is used with 869 samples processed. Furthermore, the average data freshness for DropNew is 112.4 ms (resulting in 1984 missed deadlines), whereas DropOld maintains an average data freshness of 90.7 ms (with 0 missed deadlines).

5. Discussion

The simulated results demonstrate some fundamental trade-offs in AIoT systems, namely demonstrating balancing of model accuracy, latency and data freshness under varying loads.

In the low traffic scenario of FPS=20, the system behaves optimally as expected. Since the data arrival rate (one sample every 50 ms) is lower than the processing capacity of the baseline model (40 ms), the queue never gets loaded. Here, accuracy can be prioritized (98.2%) without sacrificing data freshness (45.0 ms). However, as the arrival rate increases, the results change.

Once the FPS=50 setting is used, the scheduler switches to the pruned model on the edge device. This trades a drop in accuracy (to 97.7%) for faster execution. Despite this switch, the arrival rate (20 ms) still exceeds the pruned model's computation time. This indicates a hardware bottleneck, so when data arrives faster than the processor can clear it, the queue builds up. This leads to a high latency of 109.7 ms, 990 missed deadlines, and a degraded data freshness of 114.7 ms. It shows that model compression is not enough to stay under deadline if the underlying hardware can't keep up, and perhaps the scheduling should be implemented differently to deal with this and make sure the system does not miss the deadline, perhaps by lowering the threshold for the pruned model on cloud, so that it is used in this case instead of the quantized model which is used at even lower thresholds. Another solution is a combination of both pruned and quantized model that could be used to lower the latency. While this would sacrifice more accuracy, it might meet the deadline.

Studying the highly overloaded scenario (FPS=150), the system actually utilizes a successful distributed deployment. The system offloads the computation to the cloud server, which has higher computation speeds and can quicker clear the queue. Even though there's a network latency, it still manages to stay under the deadline of 100 ms. However, to achieve this it had to drop 70.9% of the incoming samples. Which might be accepted for some systems, and not for others depending on the objective of the system. This dropped the accuracy to 97.1%, but successfully kept the data freshness at an excellent 90.7 ms. In this case we see that 4 models are used, but mainly the pruned model on cloud is selected, with the worst accuracy of 95.3% while all other hold an accuracy

of 100%. It is expected for the pruned model on edge device to have similar accuracy, but since it was only used on 0.1% of all samples, that results is not fully reliable of that model.

These three cases clearly demonstrate the tradeoff between accuracy, freshness and number of handled samples in the system. Once the load of the system increases, one or two will be sacrificed to achieve the other.

DropOld vs DropNew

At first glance it might seem like DropNew performs better than the DropOld policy, since it processes more samples. However, it comes at a cost to data freshness. Due to DropNew rejecting incoming fresh data when the queue is full, the system is forced to process older, stale data. This traps the edge device into wasting its limited computation on evaluating old data that will miss the deadline anyway, causing model choices to fluctuate and average data freshness to go down to 112.4 ms. The unstable queue behaviour causes the model to switch between quantized model on edge and pruned on cloud. As seen before, the quantized model causes high latency due to bad scheduling.

In contrast, DropOld throws away waiting samples to make room for the newest data. While this results in more dropped samples, it drastically improves the freshness (90.7 ms). By ensuring the queue only consists of relevant data, old data is avoided, it maintains stable latency under the deadline, and consistently utilizes the cloud-pruned model. However, at the cost of dropping more samples.

Lastly the MAC's were examined, showing that the Baseline model and Pruned model both have Total Parameters: 1,199,882 and Total Mult-Adds (MACs): 12,051,08. This is likely due to the PyTorch function just zeroing out filters during pruning rather than completely removing them, meaning the dimensions don't actually change. In a real world system, this would not be the case, the number of MAC's should reduce according to LEC05. Quantization however has Total Parameters: 18,816 and Total Mult-Adds (MACs): 10,870,016, showing that the computation as well as parameters are highly reduced.

This project comes to show that the age of data, latency and accuracy are all important parameters to consider when building an AIoT system. A highly accurate prediction is entirely worthless if its Age of Information exceeds the deadline, meaning the system should be adapted to prioritize what the objective of the system actually is. If the deadline is high, the accuracy might be prioritized, while if it's low it might be suitable to drop more samples and have a slightly lower accuracy while holding the deadline. Further, implementing a sufficient scheduler is crucial, this experiment is truly simplified and can be drastically improved by using various techniques and not just studying the queue size when selecting between the models. By having further options such as a baseline model on the cloud, it can be tweaked more and adapted better to the objective of the system.

References:

- [1] PyTorch Contributors. (n.d.). *MNIST Example Repository*. GitHub. Retrieved from <https://github.com/pytorch/examples/tree/main/mnist>
- [2] PyTorch Contributors. (n.d.). *torch.optim.Adadelta — PyTorch 2.1 documentation*. Retrieved from <https://pytorch.org/docs/stable/generated/torch.optim.Adadelta.html>
- [3] Hey Amit. (2024, November 6). Mastering Model Pruning in PyTorch. *Data Scientist's Diary, Medium*. Retrieved from <https://medium.com/data-scientists-diary/mastering-model-pruning-in-pytorch-789f4b3ef0be>